

课程笔记

因果机制可解释性 Lecture2: 计算基元

五道口纳什 & Codex

2026 年 5 月 31 日



视频作者/频道: 至高机器智能

发布日期: 2026 年 4 月 10 日

视频时长: 01:19:36

视频链接: <https://www.bilibili.com/video/BV1N5QwBVE5F>

目录

1 计算基元：可解释性到底想交付什么	3
1.1 从整体能力到内部机制	3
1.2 电路、抽象和计算 motif	3
1.3 例子预告：ROME、induction heads、IOI 和更多 motif	4
1.4 本章小结	5
2 Induction Heads：最小的可复用机制样板	5
2.1 Induction head 解决什么问题	5
2.2 注意力是读写寄存器	6
2.3 为什么 induction head 是计算 motif	7
2.4 本章小结	7
3 IOI 电路：把 transformer 看成信息流图	8
3.1 Circuit analysis 的基本图景	8
3.2 Indirect Object Identification 任务	9
3.3 IOI 算法的三步结构	10
3.4 IOI 电路的边界	10
3.5 本章小结	11
4 Colored Objects：从一个任务迁移到另一个任务	11
4.1 任务形式和 IOI 的相似性	11
4.2 Path patching 与因果中介	12
4.3 两个电路的重叠和差异	13
4.4 人工干预：修复 inhibition 之后发生什么	13
4.5 这说明了什么	14
4.6 本章小结	15
5 机制迁移的限制：正确性、泛化与人工性	15
5.1 干预为什么显得人工	15
5.2 “正确实现”与语义泛化	16
5.3 机制洞见怎样转化为训练信号	17
5.4 更大模型里电路会怎样	18
5.5 更多 motif 线索	18
5.6 本章小结	20
6 从电路到权重空间：共享结构与专属结构	21
6.1 为什么只做电路分析还不够	21
6.2 共享电路到专属记忆的谱系	22
6.3 Fisher / Hessian 近似的直觉	23
6.4 K-FAC 风格的矩阵分解	24
6.5 投影、编辑与行为测量	26
6.6 本章小结	27

7 应用与开放问题：从选择性编辑到科学发现	28
7.1 选择性编辑与脆弱性谱	28
7.2 长上下文和规模化 circuit tracing	28
7.3 科学模型中的发现工具	29
7.4 本章小结	29
8 总结与延伸	30
8.1 讲者的收束	30
8.2 全讲核心链条	30
8.3 概念压缩	31
8.4 后续阅读和检查清单	31

1 计算基元：可解释性到底想交付什么

1.1 从整体能力到内部机制

本讲开场先把问题压到一个很具体的位置：做机制可解释性时，研究者到底希望从模型内部拿到什么？讲者的回答并不满足于“某一次输入为什么给出这个输出”的局部解释。他关心的是更高层的结构：模型怎样实现语言能力，怎样偏好某些内部算法，哪些结构会在不同任务、不同论文、不同模型中反复出现。

开场字幕给出了一点来源边界：讲者被介绍为 Goodfire 的 Jack，刚从 Brown 毕业，并曾自己开设过 interpretability 课程。这个背景会影响后文读法，因为第六章和第七章涉及 Goodfire 的正在进行中的工作。本文保留“Jack”这一低风险称呼；完整姓名和拼写需要依赖视频画面或外部资料进一步核验。

这里的关键词是机制可解释性。按照本讲的用法，机制可解释性研究模型内部组件如何实现行为，重点落在机制、因果路径和可复用结构上。若只看到输入和输出，研究者最多知道模型在行为层面做对或做错；若能追踪内部路径，就可以进一步问：哪些组件读入了哪些信息，信息沿哪些路径传播，哪些中间状态对最终预测有因果贡献。

讲者给出的背景判断很直接：如果可解释性只积累一袋互不相连的小发现，研究者距离理解模型仍然很远。语言模型已经能连贯地使用语言，这件事本身提出了科学问题：模型从训练数据中获得了什么样的“程序感”？它和人类语言能力相似在哪里，差异又在哪里？这些问题要求研究者从零散现象中压缩出内部结构。

本讲的交付标准

一项可解释性结果的价值，不只在解释一个样例，还在于能否抽象出可复用的计算基元。计算基元指模型反复学到并跨任务复用的小型算法构件，像程序里的基础操作。

用程序作类比，单次预测像程序的一次运行轨迹；计算基元像程序库里的基础函数。运行轨迹告诉研究者“这一次发生了什么”，基础函数告诉研究者“这类系统通常怎样工作”。机制可解释性的野心就在后者：从 transformer 的注意力头、MLP、残差流位置等组件中，找出可组合、可迁移、可复查的小型算法结构。

1.2 电路、抽象和计算 motif

要把这种野心说清楚，需要区分三个层次：电路、计算基元、计算 motif。电路指模型组件和信息流路径的组合，承载某个预测或中间计算的关键因果贡献。这里借用“电路”一词时，含义来自因果路径图，而非硬件线路或完整模型；它更像一张经过因果筛选的信息流图，图里的节点可以是注意力头、MLP 或某个残差流位置，边表示信息从一个组件流向另一个组件。

计算基元是从电路中提炼出的更小算法构件。若某类电路反复执行“从上下文中找回相关片段”“把候选答案搬到输出位置”“压低错误候选”等操作，这些操作就有机会成为计算基元。计算 motif 或计算母题则更偏经验归纳：它指跨论文、跨任务重复出现的机制模式，语气上比“基元”少一点基础操作的确定性，多一点“研究者反复看到这个图案”的意味。

三个层次的直觉

电路像一段具体线路图：哪些组件相连，哪些路径重要。计算基元像可复用的小算法：例如上下文检索。计算 motif 像研究史里的重复旋律：不同任务中反复出现的机制图案。

讲者在 00:03:08–00:03:44 附近明确提出，低层级电路分析能给出 algorithmic primitives；这些结构由训练数据压力塑造，会跨模型出现，也会被模型用来解决多类任务。这个说法很强，所以需要加上操作性约束：一个候选基元至少要经得起三种检查。第一，它要能定位到模型内部组件和路径；第二，它要能通过干预或消融显示因果作用；第三，它要能在一个窄样例之外复现，最好能跨任务或跨模型重现。

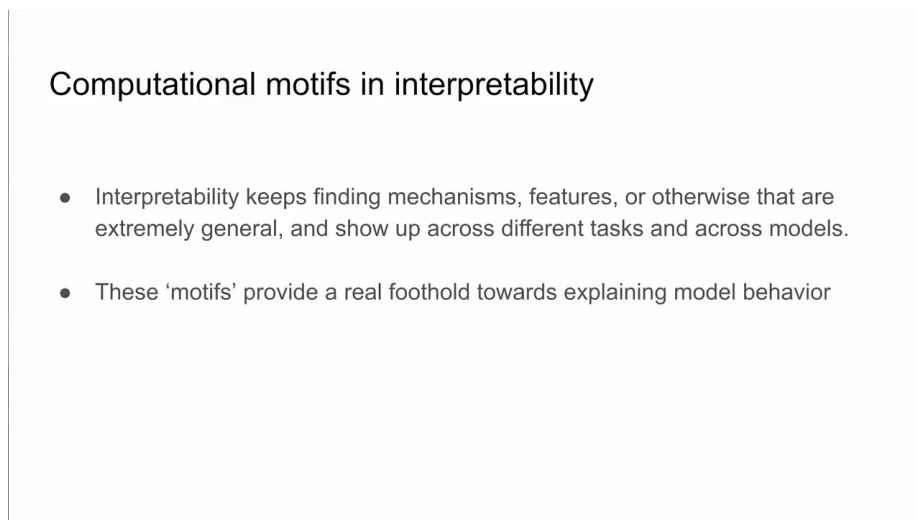


图 1: 讲者给出本讲核心论点：可解释性研究反复找到跨任务、跨模型复现的机制、特征与 motif，这些结构为解释模型行为提供了稳定支点。¹

这种检查把“看起来相似”和“机制上复用”分开了。两个任务的输出都像检索，并不自动说明它们共享电路；只有当研究者能指出共同组件、共同路径、相似中间变量，或能用路径级反事实替换（第四章会正式介绍 path patching）、消融、激活替换等方法测到因果贡献时，复用论断才开始站稳。

1.3 例子预告：ROME、induction heads、IOI 和更多 motif

讲者随后给出一组路线图式的例子。ROME 论文把一部分语言模型看成可编辑的知识库：研究者可以定位并修改模型中的事实性关联，例如让模型把某个地标的位置联想到另一个城市。这个例子展示的是“模型内部可能有可编辑的知识结构”。

第二个例子是 induction heads，本讲采用术语“归纳头”。归纳头是一类注意力头，会在上下文中发现重复模式，并把前文相似位置之后的信息带到当前位置。它是本讲最小、最清楚的可复用机制样板，因为它把一个抽象操作压缩得很干净：在当前上下文里发现“曾经出现过类似前缀”，再把当时后续的 token 信息拿来辅助下一 token 预测。

第三个例子是 IOI 电路，即间接受词识别电路。讲者把它放在归纳头之后，是因为 IOI 的核心已经扩展到多组件信息流图：不同注意力头和中间表示协作，让模型在 John/Mary 这类句子中预测正确的间接受词。后续章节会进一步说明，IOI 的价值在于它提供了一个可检验的电路分解，也暴露了任务边界。

这张路线图也给读者一个判断框架：ROME 指向可编辑知识，归纳头指向上下文模式检索，IOI 指向多组件电路分析，后面的 colored objects 和权重空间分解会继续追问这些结构能迁移到哪里。

¹视频片段：00:03:18–00:03:44。

²视频片段：00:05:12–00:05:45。

Outline

- Quick induction head review (a classic computational motif)
- Motifs in the “Indirect Object Identification” circuit
- More examples of motifs
- Challenges and open problems

图 2: 讲座路线图把后续内容分成 induction head 回顾、IOI 电路中的 motif、更多 motif 示例以及开放问题, 先给读者一张全局索引。²

换句话说, 本讲没有按论文列表堆材料; 它沿一条线推进: 从一个机制实例, 到跨任务复用, 再到更大范围的共享结构。

不要把“解释一个任务”放大成“解释整个模型”

讲者的主张是模型内部存在可复用机制线索, 并没有声称某一个电路能解释所有语言能力。电路解释有任务边界、数据分布边界和模型规模边界; 后续 Q&A 会专门给这个乐观判断降温。

1.4 本章小结

本章建立了全篇的概念合同。机制可解释性要交付的核心成果, 是对模型内部程序的可复用理解。电路给出具体组件和信息流路径, 计算基元把电路中的小算法抽象出来, 计算 motif 记录跨任务、跨论文反复出现的机制模式。

这套术语的实际用处在于压缩复杂性。若每个 prompt 都只能得到一份孤立解释, 研究者面对的仍然是海量个案; 若能找到归纳头、IOI 类电路、上下文检索、抑制错误候选这类反复出现的结构, 模型内部就开始像一个可以拆解的程序系统。下一章的归纳头正是这个思路的最小样板。

2 Induction Heads: 最小的可复用机制样板

2.1 Induction head 解决什么问题

归纳头解决的是上下文中的重复模式延续问题。讲者给出的例子可以写成:

A B C A B C A B ?

模型若已经在同一上下文中见过 A B C, 当当前位置再次出现 A B 时, 一个合理预测是 C。归纳头的机制直觉是: 当前的 B 去寻找前文相似模式中的 B, 再把那个旧 B 后面的 C 信息带到当前位置, 帮助模型预测下一个 token。

按照术语表，归纳头是一类注意力头，会在上下文中发现重复模式，并把前文相似位置之后的信息带到当前位置。这里的“归纳”含义很窄：它不涉及哲学里的完整归纳法，也不涉及训练阶段重新学习规则；它是在一次前向推理的上下文窗口内，把前文模式临时当作可用证据。

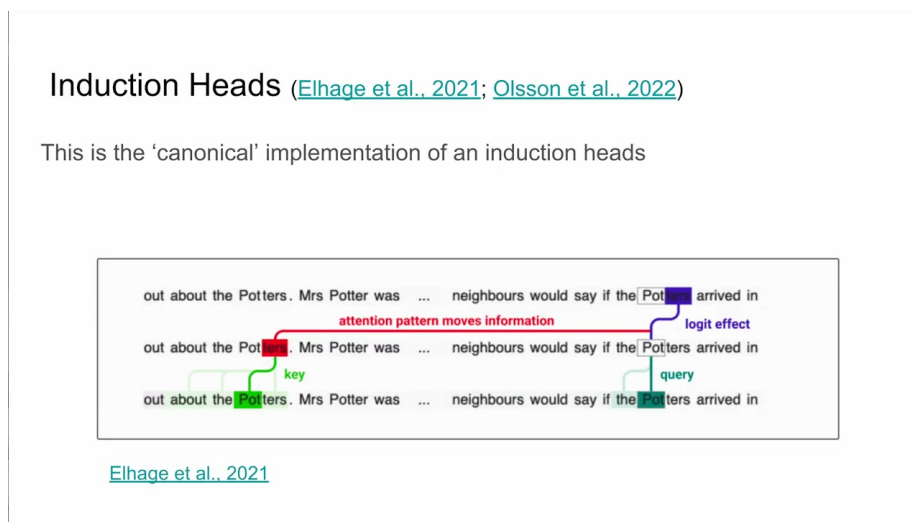


图 3: Induction head 的标准图示：注意力模式把前文相同 token 后面的信息搬运到当前位置，形成可复用的序列续写计算基元。³

这个例子小得像玩具，但它的教学价值很高。它把上下文学习拆成了三个可检查步骤：先匹配相似前缀，再定位前文后继 token，最后把后继 token 的信息用于当前位置预测。只要这三步能在注意力图、残差流中的写入或干预实验中被观察到，研究者就不必把“模型会续写重复序列”停留在行为描述上。

2.2 注意力是读写寄存器

讲者用“read and write register”描述注意力，这个类比很适合解释 transformer 中的局部信息搬运。注意力头是 transformer 中读取其他 token 位置并写入当前表示的子模块。残差流指 transformer 层间持续传递的向量通道，各模块读入并写回该通道。把两者放在一起看，一个注意力头就像一个小读写单元：它从若干历史位置读信息，把加权后的结果写回当前位置的残差流。

这个过程可以用一个简化公式表示。对当前位置 i ，一个注意力头输出：

$$\text{Attn}(x_i) = \sum_{j \leq i} \alpha_{ij} V x_j, \quad \alpha_{ij} = \text{softmax}_j \left(\frac{(Q x_i)^\top (K x_j)}{\sqrt{d_k}} \right)$$

符号说明：

- x_i ：当前位置的 token 表示，属于当前层读入的向量状态。
- x_j ：第 j 个历史位置的 token 表示。
- Q, K, V ：注意力头内部的查询、键、值线性映射。
- α_{ij} ：当前位置 i 对历史位置 j 的注意力权重。
- d_k ：查询和键向量的维度，用于缩放点积；本文把 d 保留给后文的权重方向。

³视频片段：00:06:37–00:06:52。

在归纳头里，关键点落在 α_{ij} 的指向：它学会了选择“前文相似模式的后继位置”。讲者特别提醒，注意力图上的箭头方向和信息流方向容易让人混淆：当前 token 可以注意到前文 token，但被读出的信息会写入当前位置，随后沿序列向前影响下一 token 预测。也就是说，注意力权重图描述“从哪里读”，残差流更新描述“写到哪里并继续传播”。

常见混淆：注意力权重不等于完整机制

看到一个 token 注意到另一个 token，只说明读信息的候选路径存在。要把它称为归纳头，还需要说明它读到了什么、写回了什么、这些写回信息怎样影响后续预测，并最好通过干预证明这条路径有因果贡献。

2.3 为什么 induction head 是计算 motif

归纳头之所以在本讲中承担样板角色，原因在于它小、清楚、可迁移。它的解释范围超出了一个固定字符串；讲者提到，相关论文把归纳头和 in-context learning 联系起来，并且后续大量研究都围绕 induction heads 展开。这个事实说明，研究者反复在不同语境中看到类似的上下文检索机制。

把归纳头称为计算 motif，需要把“功能相似”和“结构复用”同时放在桌面上。功能层面，它做的是上下文检索：从当前 prompt 的前文中找回相关信息并用于预测。结构层面，它通常依赖注意力头对历史位置的选择、位置信息的传递，以及残差流中可被后续组件读取的写入结果。若把模型看成信息处理系统，归纳头就像一个局部缓存查找器：当前查询先命中前文的相似键，再取回键后面的值。

归纳头的机制压缩

归纳头把“模型能利用上下文重复模式”压缩成一个可复查机制：匹配前文相似位置，读取该位置之后的信息，把结果写入当前位置的残差流，并改变下一 token 的预测。

这种压缩对后续章节很关键。IOI 电路会问：多组件电路能否像归纳头一样被拆成稳定的信息流路径？Colored objects 会进一步问：一个任务中的电路组件，能否迁移到另一个看似不同的上下文检索任务？归纳头在这里提供了类比基准：如果一个小机制可以反复出现，那么更复杂的电路也可能包含可复用部件。

2.4 本章小结

归纳头是本讲的第一个具体机制样板。它从一个简单问题出发：当前上下文中出现了重复模式，模型怎样利用前文后继 token 预测当前位置之后的内容？答案落在注意力头和残差流的配合上：注意力头从历史位置读信息，写回当前位置表示，后续组件再把这份信息转成下一 token 倾向。

本章要留下的判断是：归纳头的意义超过了一个玩具序列例子。它展示了计算基元应当具备的形态：内部路径可定位，因果贡献可测试，机制能跨场景复用。后面的 IOI、colored objects 和权重空间分析，都会继续沿着这个标准追问模型内部是否存在更大的可复用结构。

3 IOI 电路：把 transformer 看成信息流图

3.1 Circuit analysis 的基本图景

沿用第一章的定义，本章把电路分析具体化为一张有向信息流图。图里的节点是模型内部的组件，例如某一层注意力头、MLP、残差流中的某个 token 位置；图里的边是信息从一个组件流向另一个组件的通道；一条路径则是多条边串起来的机制链。电路分析要找的，是这张图中对某个预测真正有因果贡献的节点、边和路径。

这个视角把 transformer 从“巨大矩阵堆叠”改写成“信息如何被读取、搬运、抑制、写回”的工程图。注意力头像临时读写器：它从一些位置读信息，把结果写回残差流；后面的组件再从残差流中读到这个结果。残差流 h 可以直观地看成一条贯穿层间的公共总线，每个模块都在这条总线上读写：

$$h_{\ell+1} = h_{\ell} + \text{Attn}_{\ell}(h_{\ell}) + \text{MLP}_{\ell}(h_{\ell})$$

其中：

- h_{ℓ} ：第 ℓ 层之前的残差流状态；
- Attn_{ℓ} ：第 ℓ 层注意力模块写入残差流的信息；
- MLP_{ℓ} ：第 ℓ 层 MLP 模块写入残差流的信息；
- 加号表示这些组件的输出被累积到同一条表示通道中，后续组件可以继续读取。

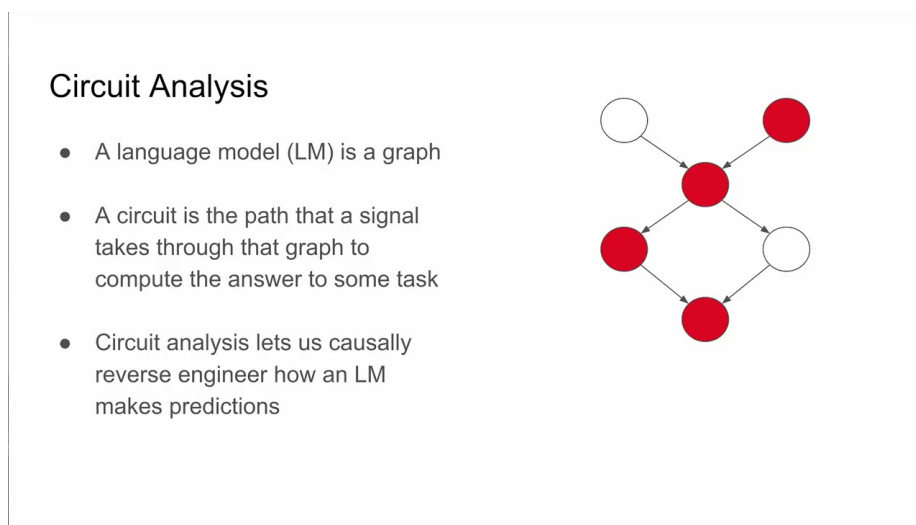


图 4: Circuit analysis 将语言模型视为由组件节点和信息流边构成的图，电路就是完成某个预测所需的高影响路径。⁴

电路资格来自因果贡献

进入电路的资格来自因果贡献，而非激活大小或 attention heatmap 的醒目程度。若移除无关节点后预测基本不变，这些节点就缺少进入关键电路的资格；若替换某条路径上的中间状态会系统性改变目标 logit，这条路径才有资格进入机制解释。

⁴视频片段：00:08:54–00:09:12。

这里有一个容易混淆的点：attention heatmap 只能说明某个头“看向哪里”，不能自动说明“它为什么重要”。电路分析关心的是因果贡献。一个头可以有很醒目的注意力模式，却对最终 token 概率贡献很小；另一个头的权重图看起来平淡，却可能在残差流里写入后续组件必需的方向。讲者强调“causally reverse engineer”的含义就在这里：解释要通过干预来检验，不能只靠视觉相关性。

3.2 Indirect Object Identification 任务

IOI 是 Indirect Object Identification，即“间接受词识别”。典型 prompt 类似：

John and Mary went to the store. John gave a drink to ____

在这个例子中，合理的下一个名字是 Mary。研究者把问题收窄到一个判别式目标：为什么模型给 Mary 的 logit 高于 John 的 logit？也可以写成：

$$\text{logit}(\text{Mary}) > \text{logit}(\text{John})$$

其中：

- $\text{logit}(t)$ ：softmax 之前给 token t 的未归一化分数；
- Mary：当前句式中的目标间接受词；
- John：句中重复出现、但在结尾处不应再次作为受词出现的名字。

这个任务看起来很小，甚至带有合成数据集的味道。讲者也直说，大家并不真的只关心 IOI 这个具体行为。它的重要性来自另一个层面：如果研究者能在这样一个小任务上把模型内部的路径拆清楚，就有机会问下一层问题：这些路径会不会复用到别的上下文检索任务中？

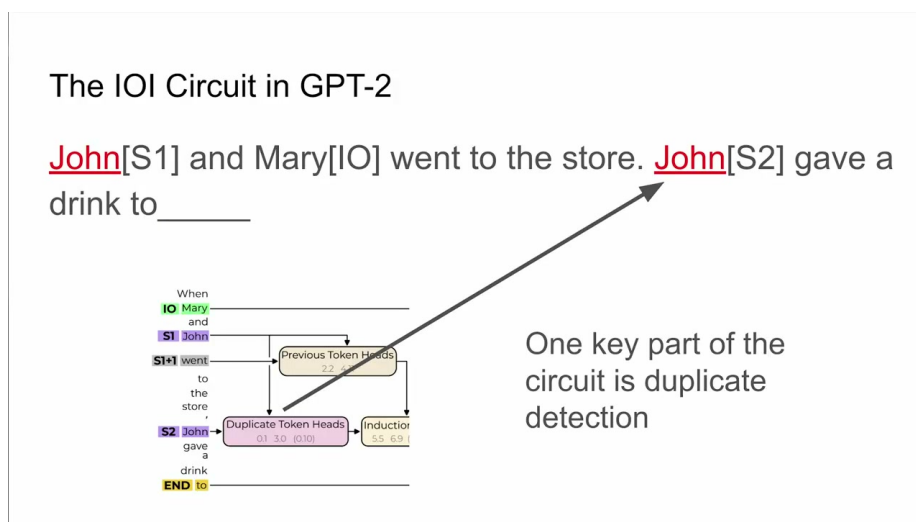


图 5: IOI 例子中的重复检测子电路：模型需要识别 John 的重复出现，进而在最终位置抑制重复姓名、提升间接宾语 Mary。⁵

⁵ 视频片段：00:13:07–00:13:14。

IOI 的边界要写窄

IOI 电路解释的是特定数据分布上的间接受词预测，不等同于完整指代消解，也不等同于通用句法理解。它是一个可检验的小窗口，用来观察 transformer 如何搬运名字、位置和抑制信号。

3.3 IOI 算法的三步结构

讲者把 IOI 电路概括成三步。第一步，模型识别重复名字。在例句中，John 出现了两次，模型需要记录“John 是重复项”这个事实。这个阶段会涉及重复 token 头、归纳头一类组件，它们传递的更像是位置和重复关系，并非最终答案本身。

第二步，模型把“不要再说 John”的信号传给后面的组件。课程里把这类组件称为 inhibition heads，即抑制头。抑制头从最后位置出发，注意到重复的名字位置，向残差流中写入一种方向：后面的搬运头应该少看 John。直觉上，这像在候选名单上给重复候选贴一个“降权”标签。

第三步，mover heads，也就是搬运头，把被注意到的名字信息搬到输出位置。如果搬运头看向 Mary，它就会提升 Mary 的 logit；如果看向 John，它就会提升 John 的 logit。抑制头的作用在于改变搬运头的注意力分配，让它少看错误候选，多看正确候选。用一个粗略的路径图表示，就是：

重复名字线索 → 抑制信号 → 搬运头注意力 → 目标名字 logit

其中：

- 重复名字线索：John 出现两次这一结构性事实；
- 抑制信号：由 inhibition heads 写入的“降低重复候选”的中间表示；
- 搬运头注意力：mover heads 在候选名字之间分配注意力；
- 目标名字 logit：最终对 Mary 或 John 的输出分数。

为什么“搬运头”这个名字有用

搬运头这个译法保留了机制直觉：它不负责理解完整句义，也不负责生成最终句子；它更像一个定向搬运器，把某个 token 的表示搬到输出位置并提升相应 logit。这个类比有局限，但能帮助读者把“注意力权重图”与“输出 token 分数”连接起来。

这三步说明了电路分析的风格：它不会停在“模型知道 Mary 是答案”这种行为描述上，还会继续追问模型内部如何把重复检测、候选抑制和名字搬运接起来。机制解释的粒度也因此变细：一个任务行为可以被拆成多个局部计算，每个局部计算由若干组件和路径承载。

3.4 IOI 电路的边界

讲者紧接着指出 IOI 电路的脆弱边界。若在原句中插入一句“Mary was having a great time”，John 和 Mary 都出现两次，原先依赖“重复名字”来抑制 John 的电路就不再完整解释行为。也就是说，电路在原数据集上可以很准确、很细致，但它解释的仍然是一个特定 prompt 分布。

这并非对 IOI 工作的否定。恰恰相反，它把机制可解释性的评判标准推高了：一个电路若只解释一个狭窄数据集，它仍然有局部价值；若它的组件能跨任务复用，它才更接近“计算基元”。因此，

讲者在 00:14:34–00:14:58 处提出后续问题：别的任务会不会使用类似电路？这些看起来很通用的功能部件，是否也像归纳头一样反复出现？

从 IOI 到计算基元的问题转向

IOI 的教学价值在于提供一个可干预、可复查的电路分解；玩具任务只是承载这个分解的实验场景。随后要检验的是：重复检测、抑制、搬运、上下文检索这些部件能否迁移到另一个任务。

3.5 本章小结

本章把 transformer 暂时视为信息流图：节点是组件，边是信息通道，路径是承载预测所需信息的机制链。IOI 任务给出了一个小而清楚的观察窗口：模型先识别重复名字，再通过抑制头降低错误候选，最后由搬运头提升正确名字的 logit。

同时，IOI 的解释边界也必须保留。它不能充当通用语法解析器，也不能直接代表所有指代消解。它更像一张局部机制地图：地图本身有价值，但更大的问题是这张地图上的道路能否通向别的任务。第四章的 colored objects 正是这个迁移测试。

4 Colored Objects：从一个任务迁移到另一个任务

4.1 任务形式和 IOI 的相似性

讲者在 Big-Bench 风格任务中选出 colored objects 作为迁移对象。任务形式很简单：prompt 列出若干带颜色的物体，例如 blue pencil、black necklace、yellow lighter，然后询问 “What color is the pencil?”。模型需要从当前上下文中找回 “pencil” 对应的颜色 “blue”。

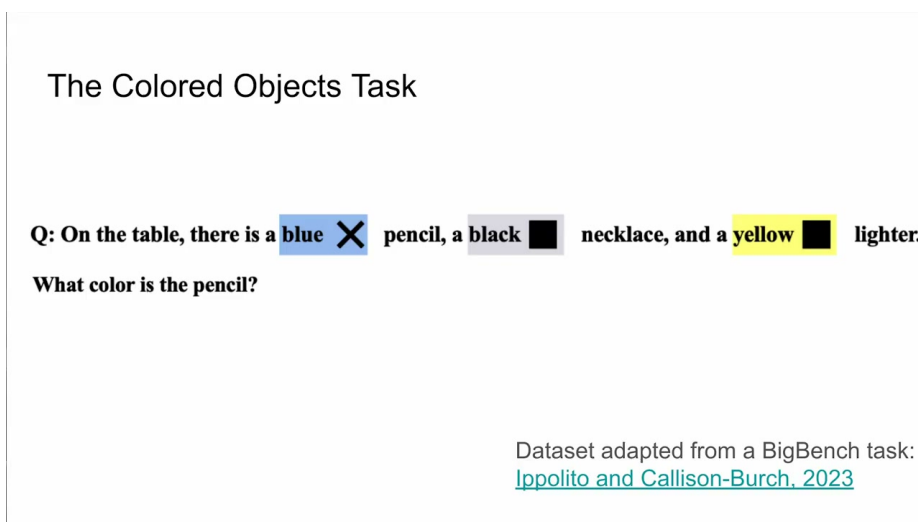


图 6: Colored Objects 任务把多个带颜色的物体放进同一上下文，再询问某个物体的颜色；它提供了一个与 IOI 类似的上下文检索测试场景。⁶

这与 IOI 在表面语义上差异很大：一个讲人物和动作关系，一个讲物体与颜色配对。可是从计算形式看，二者都要求模型做上下文检索。IOI 要从前文候选名字中找回目标名字；colored objects 要从前文物体列表中找到目标物体对应的颜色。抽象一点说，二者都在做键值检索：

⁶视频片段：00:15:07–00:15:23。

$$\text{query} = \text{目标线索}, \quad \text{context} = \{(\text{key}_i, \text{value}_i)\}_{i=1}^n$$

其中：

- query：当前问题给出的检索线索，例如 pencil；
- key_i ：上下文中的对象名，例如 pencil、necklace、lighter；
- value_i ：与对象绑定的答案，例如 blue、black、yellow；
- 检索目标：找到与 query 匹配的 key，并把对应 value 提升为输出。

“相似任务”和“相似电路”的区别

两个任务在自然语言表面上相似，并不保证内部电路相似；两个任务在语义主题上不同，也可能复用相近的上下文检索组件。机制可解释性真正关心的是后者：模型内部是否使用了相似的节点、边和路径。

4.2 Path patching 与因果中介

为了比较 IOI 与 colored objects 的电路，讲者使用 path patching。可以把它理解为沿某条信息路径做反事实干预：先让模型在一个干净输入和一个对照输入上运行，再把某个早期组件到某个后期组件之间的信号替换掉，观察最终输出怎样变化。

它与因果中介分析的关系很直接。因果中介分析问的是：原因 X 对结果 Y 的影响，有多少通过中间变量 M 传递？path patching 在模型里问类似问题：某个上游组件 A 对最终 logit Y 的影响，有多少通过下游组件 B 或某条路径 $A \rightarrow B$ 传递？

$$\text{PathEffect}(A \rightarrow B) = Y(\text{patch}(A \rightarrow B)) - Y(\text{baseline})$$

其中：

- A ：被考察的上游组件，例如重复 token 头或抑制头；
- B ：被考察的下游组件，例如 mover head；
- Y ：目标输出指标，可以是正确答案 logit、错误答案 logit 或二者差值；
- $\text{patch}(A \rightarrow B)$ ：只替换指定路径信号的反事实运行；
- baseline：未做该路径替换时的模型运行。

Path patching 的定位

Path patching 的对象并非普通 prompt 改写，也并非某层激活大小。它是一种模型内部的反事实实验：固定大部分计算，只替换某条路径上的中间信号，用输出变化估计这条路径的直接因果贡献。

这个方法之所以重要，是因为“两个任务都能答对”无法说明它们用了同一套机制。“两个任务都有类似注意力图”也不够。只有当替换特定路径会以相似方式改变输出，研究者才有更强理由说：这里可能存在共享电路组件。

⁷ 视频片段：00:18:48–00:19:06。

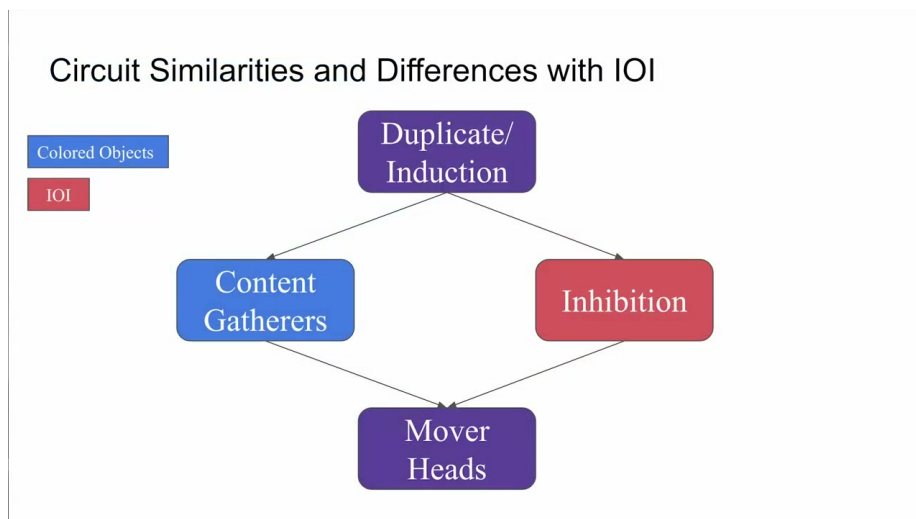


图 7: Colored Objects 与 IOI 电路共享 duplicate/induction、content gatherers 和 mover heads 等结构，同时在 inhibition 组件上出现关键差异。⁷

4.3 两个电路的重叠和差异

讲者总结 path patching 结果时说，IOI 与 colored objects 在做上下文检索时有大量相似电路。课程契约与字幕都记录了一个约 78% overlap 的说法。这里需要谨慎措辞：F08 展示的是共享组件类别和主要差异，并没有给出可读的数值表格；因此 78% 在本文中只作为“讲者口头报告的重叠量级”，不能写成未经审查的精确定理。

差异集中在 inhibition heads。IOI 里，抑制头对“不要复制重复名字”有清楚的因果作用；colored objects 中，讲者说这些抑制头没有同样激活，或者虽然有活动，却没有在 colored objects 预测中承担同样的因果角色。换句话说，两个任务共享了很多上下文检索部件，但 colored objects 缺了 IOI 中最关键的一段抑制逻辑。

关于 78% overlap 的审慎读法

78% 应作为当前字幕中的经验报告，不能独立支撑“两个任务本质相同”这类强结论。更稳妥的结论是：讲者展示了大量重叠证据，同时指出 inhibition heads 是主要差异点；精确数值需要原论文或更清晰结果表格校准。

这个差异很有教学价值。若 colored objects 完全照搬 IOI 电路，结论会显得机械；若二者毫无重叠，计算基元的主张又会变弱。现在看到的是更真实的形态：共享结构与任务专属结构混在一起。一些搬运、检索和位置相关组件可以复用，某些抑制路径则取决于任务需要和模型是否学会了相应控制。

4.4 人工干预：修复 inhibition 之后发生什么

接下来讲者做了一个更直接的实验：既然 colored objects 中 inhibition heads 没有发挥 IOI 中那种作用，那就人工把它们的注意力模式调向更合适的位置，看看模型和电路剩余部分如何响应。字幕在 00:21:15–00:21:45 记录到，经过少数选定 head intervention 后，任务表现从约 50% 提升到约 94%。这个提升仍需要更清晰图表或论文复核；更关键的实验意图是同时检查电路其他信号是否也朝 IOI 式机制靠近。

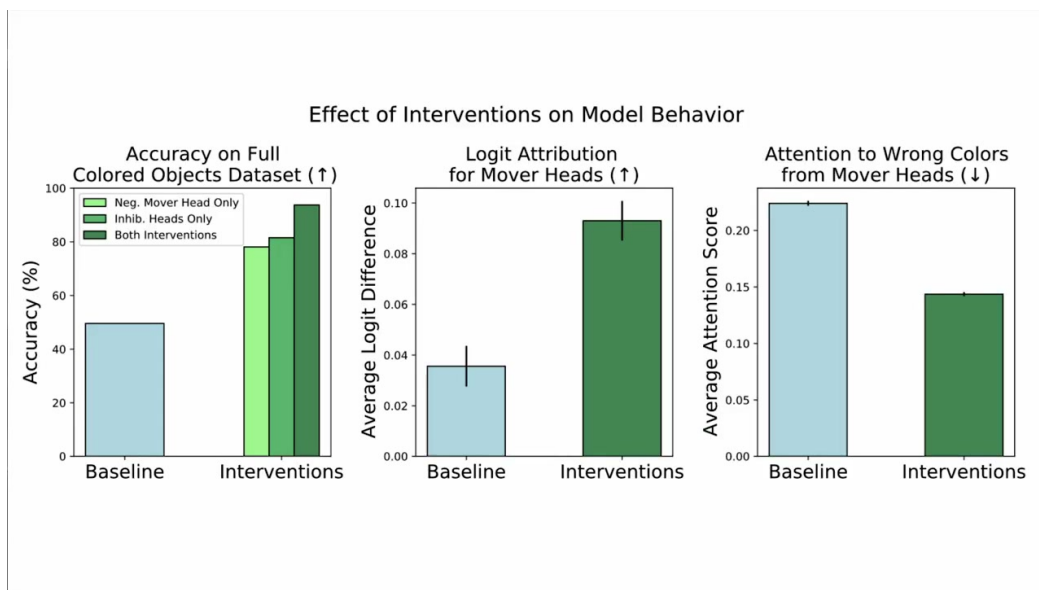


图 8: 对 inhibition heads 与 negative mover heads 的干预同时提升整体准确率、增强 mover heads 的 logit attribution, 并降低 mover heads 对错误颜色的注意力。⁸

更关键的观测是下游响应。讲者提到, 干预后 mover head 对错误答案的注意力下降, 这正是 IOI 中 inhibition heads 会造成的效果。也就是说, 研究者没有重训练整个模型, 也没有重写 prompt; 他们只是修改少数内部注意力模式, 然后观察到后续组件按预期改变。这个实验把“colored objects 缺少抑制控制”从相关性说法推进到因果说法。

人工修复实验的因果含义

若修复 inhibition heads 的注意力后, mover heads、错误答案概率和最终准确率都朝预期方向变化, 那么 inhibition 路径就更像一个可插拔的控制部件。它尚不能构成自然泛化的完整证明, 却能作为共享电路组件可能跨任务工作的强证据。

这里也要保留限制: 这种干预带有合成性。实验者知道该把注意力指向哪里, 并直接改内部模式; 真实模型在新任务中未必会自动找到同样的修复方式。因此, 本实验更适合证明“组件有跨任务可用性”, 不适合证明“模型已经稳健掌握了所有相关任务”。

4.5 这说明了什么

colored objects 的结论可以压缩成一句: 看似不同的任务, 可能共享一批电路组件; 因此, 可解释性有机会把海量行为压缩成更少的机制模块。这个压缩非常重要。若每个 prompt 都需要一张全新的电路图, 可解释性会退化成无穷尽的个案标注; 若许多任务共享上下文检索、搬运、抑制、位置绑定等计算基元, 研究者就能建立一套更小的内部机制词汇表。

这也解释了为什么讲者从 induction heads 过渡到 IOI, 再过渡到 colored objects。归纳头说明一个机制可以在不同上下文中复用; IOI 给出可细拆的电路地图; colored objects 则测试这张地图中的某些道路能否跨任务使用。三者组合起来, 形成了“计算基元”主张的早期证据链。

⁸ 视频片段: 00:21:35–00:21:58。

一个读者可能忽略的层次

“模型会做 colored objects”是行为层判断；“模型通过上下文检索做 colored objects”是算法层判断；“模型复用 IOI 中的若干组件做 colored objects”是机制层判断。机制可解释性的难点在第三层，因为它要求组件、路径和干预证据同时对齐。

4.6 本章小结

本章把 colored objects 当作 IOI 之外的迁移测试。它表面上是颜色问答，机制上是上下文检索：从对象名找到对应颜色，再把颜色 token 推到输出位置。Path patching 提供了反事实中介逻辑，用来比较早期组件到后期组件的直接因果作用。

当前字幕与课程契约显示，讲者报告了 IOI 与 colored objects 电路的大量重叠，并口头给出约 78% 的重叠量级；本版把它作为量级线索处理，不把它写成精确复现实验结果。主要差异在 inhibition heads：IOI 中它们负责压低错误候选，colored objects 中缺少同样的因果作用。人工修复这些抑制头后，mover head 注意力、错误答案概率和准确率出现预期变化，这支持“跨任务共享组件”的判断，同时也提醒读者：人工干预结果不能直接等同于自然泛化。

5 机制迁移的限制：正确性、泛化与人工性

5.1 干预为什么显得人工

上一章的彩色物体实验给出一个很强的信号：IOI 电路里学到的抑制头、搬运头和上下文检索路径，为解释另一个表面上完全不同的任务提供了强证据。讲者随即在问答中降温：这类实验说明了机制迁移的可能性，却没有自动解决“怎样把它变成通用模型编辑”的问题。

上一章已经给出合成性干预的定义；这里补上它为什么不能直接变成通用编辑。研究者在模型内部直接设定或修复某个状态，比如把注意力模式指向研究者已经知道的错误颜色项，然后观察后续电路会不会按预期恢复。它像是在钟表里用镊子拨动一个齿轮：如果后面的齿轮跟着动，说明这条传动链很可能存在；但这并不意味着钟表在自然运行时会自动学会这种拨法。

讲者对这个实验的表述很谨慎。一方面，干预很小，只是在具体注意力头上做“温和”的方向性修复；另一方面，它又非常“贴脸”，因为研究者直接告诉这些头应该到哪里取 value vector、应该把抑制信号放在哪些颜色上。换句话说，实验避免了最粗暴的做法：直接从残差流里扣掉 black、yellow 之类错误答案的 logit vector。那种做法会让模型不再预测这些 token，因果解释价值很低。当前干预更细，因为它试图修复电路中的一个小部件，再检查剩余电路是否自行接上。

人工干预的常见误读

彩色物体实验里的干预不能被解读成“已经找到可泛化的模型修补按钮”。它更接近一个受控反事实实验：研究者把一个内部信号推到指定位置，然后检验下游电路是否产生预期因果效应。这个结论可以支持电路结构假说，但还不足以支持任意新任务上的自动泛化。

讲者还明确提到“no free lunch”。如果把这种修复做成真实模型编辑，它可能会损害其他 benchmark。原因很直接：抑制头并非只为彩色物体任务服务，它也可能参与列表复制、逗号后的连词生成、长列表检索等行为。把一个共享组件强行调大，可能会让某些任务受益，也可能让另一些任务

⁹ 视频片段：00:32:10–00:33:31。

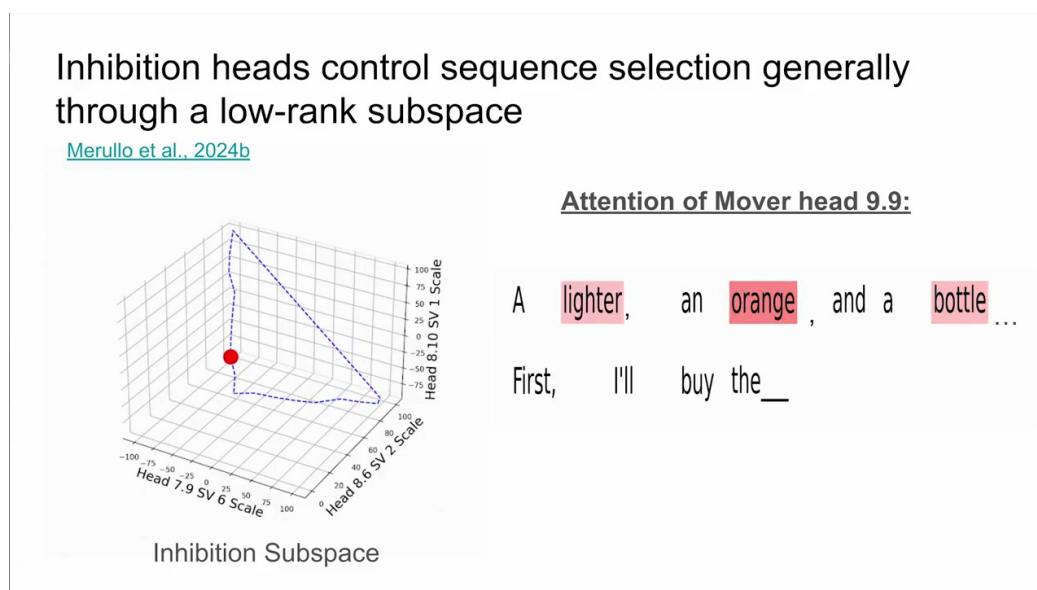


图 9: 低秩 inhibition subspace 图把多个 inhibition head 方向压缩到可视化空间中, 用来说明这些头如何调控 mover head 的序列选择行为。⁹

过度抑制。深度学习里的朴素经验是不要轻易和梯度下降硬碰硬：模型通过训练找到的折中，往往已经编码了很多人类难以手工枚举的约束。

本段结论

合成性干预的价值在于证明“修复一个局部信号，剩余电路会按因果链响应”。它的边界在于：研究者事先知道该修哪里、往哪里修，以及如何测量修复后的电路反应。

5.2 “正确实现”与语义泛化

问答里最重要的一组问题，是把“模型把题做对”和“模型用人类期待的语义方式做对”分开。彩色物体和 IOI 都含有列表检索结构：前文给出若干候选项，后文要求找回其中一个。模型可以通过抑制某些位置、移动某些 token 信息来完成任务；这种策略在许多分布上有效，却未必等同于人类直觉里的语法解析或语义理解。

听众提出的 John/Mary 例子很典型。人们可能期待模型构造出类似 parse tree 的结构：谁把东西给了谁，谁是动作主体，谁是间接受词，语义关系能跨越更长句子和更多插入成分而保持稳定。讲者承认，IOI 电路显示出的机制看起来更“笨”：它更像是在候选列表里抑制某些名字，再把剩余候选搬到输出位置。这个机制可以解决一批句式，却不保证覆盖更丰富的语言结构。

这里要避免一个价值判断陷阱。“正确实现”至少有两层含义。第一层是行为正确：模型给出正确 token，损失下降，准确率提高。第二层是机制正确：内部算法能否按人类认为合理的语义变量组织信息，并在句式变化、长度变化、插入成分变化时继续工作。两层经常相关，却不能互相替代。一个简短公式可以表达这种差别：

行为成功 $\nRightarrow$ 语义机制成立

其中各符号的含义是：

- 行为成功：模型在当前数据分布上给出正确答案，或某个 benchmark 指标提升。

- 语义机制成立：模型内部表示明确承载语义角色、句法关系或可迁移变量，并能支持分布外变体。
- $\nrightarrow$ ：单向不蕴含；它只说明前者不能单独证明后者。

不要把“做对”写成“理解了”

本讲支持的说法是：电路分析能揭示模型完成任务时使用的内部路径。字幕没有支持更强说法，例如模型已经获得人类式语义解析，或者某个 IOI 风格电路就是所有指代消解的通用算法。

更有意思的是，讲者也没有把人类直觉当成绝对标准。他提出一种反向可能：也许语言模型用了看似粗糙的机制，也许人类对自身解析过程的想象同样过于理想化。机制可解释性的科学价值正在这里显现：它把“模型是否像人一样理解”改写成更可检验的问题，即模型内部到底有哪些变量、路径和压缩结构。

语义泛化的判据

机制泛化不能只看一个 prompt 家族上的准确率。更可靠的检查包括：句式扰动后电路是否仍然承担同一因果角色，候选项增多后子空间是否仍能区分位置，语义变量改变后同一组件是否还能按预期传递信息。

5.3 机制洞见怎样转化为训练信号

在彩色物体实验之后，听众追问了一个工程上更尖锐的问题：既然研究者知道模型内部哪里出错，接下来能做什么？讲者的回答没有把机制发现直接写成万能修补方案，但给出了清楚的中间路径：机制分析可以把“某类错误”改写成“某组表示弱点”。一旦知道模型在哪个子任务族、哪个位置检索模式、哪个候选抑制方向上容易失败，研究者就可以设计更有针对性的训练数据，而非只把更多随机样例塞给模型。

这里的关键词是 representational weak spot，可以译作“表示性弱点”。它指模型内部表征空间中没有稳定分开的区域：例如列表中间项容易混在一起，或者某个抑制方向对多个候选项区分不足。若错误只是输出层上的一个 token 偏差，补数据可能像在表面刷漆；若错误来自表示性弱点，训练数据就需要逼迫模型在相应内部变量上拉开距离。用一个简化目标写，就是让训练样例增加目标变量 z 与干扰变量 u 的可分性：

$$\Delta = \mathbb{E}[\text{sep}(z, u) \mid \text{targeted examples}] - \mathbb{E}[\text{sep}(z, u) \mid \text{baseline examples}].$$

其中 $\text{sep}(z, u)$ 表示目标变量和干扰变量在内部表示中的分离程度， $\Delta > 0$ 表示 targeted examples 更能推动这种分离。这个公式只是帮助读者理解“靶向训练数据”的含义；讲者没有在字幕中给出正式训练目标。

讲者举到的直觉可以从 colored objects 推到更日常的语言行为。比如 Oxford commas 这类逗号与列表结构问题，表面上只是标点风格，内部却可能依赖模型怎样识别列表边界、怎样压低错误候选、怎样在长序列中保持项目身份。机制洞见的价值在这里出现：它帮助研究者生成更像“压力测试”的数据，专门挤压模型薄弱的内部变量，而非只追求更多同分布样例。

“so what” 的回答

机制解释的工程价值，首先是诊断和数据设计：把行为错误定位到表示性弱点，再构造 targeted training data 去强化那一类变量。讲者同时强调，理解模型本身也有独立科学价值；并非所有可解释性结果都必须立刻折算成模型性能提升。

5.4 更大模型里电路会怎样

模型规模带来另一层限制。讲者谈到较大模型时给出的判断很克制：更大的 GPT-2 变体里会出现更多同类注意力头，机制更分布式，也可能有更多冗余路径。直观地说，小模型像一条狭窄流水线，某一层某个头错过了传递机会，后面可能就没有补救空间；大模型有更多层、更多头、更多并行通道，同一类行为可能有更多次“接球”的机会。

这个说法可以用一个简化图景表达。设某类机制需要在前向传播中至少一次成功传递信号，若每个可用组件成功传递的概率为 p_i ，多个冗余组件共同给出的成功概率可写成：

$$P(\text{至少一次成功}) = 1 - \prod_i (1 - p_i)$$

其中各符号的含义是：

- p_i ：第 i 个相关组件完成某个信号传递的概率或有效性，这里只作为直觉变量。
- $\prod_i (1 - p_i)$ ：所有候选组件都未能传递该信号的情形。
- $1 - \prod_i (1 - p_i)$ ：存在至少一条可用路径的概率。

这个公式只是帮助理解“冗余”一词，不能当作讲者给出的实证模型。字幕支持的核心意思是：大模型可能有更多同类 head、更分布式的实现、更高容错性；同时，讲者没有给出可外推到所有大模型的定论。

问答里还提到 Pythia checkpoint 一类工作，研究者可以观察电路在预训练过程中的形成。讲者记得有相关论文，但对模型规模和具体结论没有完全确认。因此本章只能把它写作研究线索：用训练过程中的多个 checkpoint 追踪电路形成，可能帮助回答机制何时出现、何时强化、是否随规模变得更抽象。

规模外推要保守

小模型里看到的电路不应直接外推成大模型里的同构电路。更稳妥的表述是：跨规模机制相似性有一些证据，特征和电路可能显示 universality；但随着模型变大，表示会更丰富、更分布式，原来的节点级解释可能需要升级为子空间、特征族或多路径解释。

5.5 更多 motif 线索

讲者随后快速列出若干计算 motif。它们共同说明一件事：语言模型在无监督预训练中，会形成比单个样例解释更丰富的内部结构。这里的 motif 不宜写成严格定理，它更像生物学里的“重复结构”：在不同物种身上反复出现，提示底层功能约束相似。

这些 motif 的发现过程也需要写清楚。讲者强调，很多 circuit-level analysis 非常细，发现过程依赖人工设计的 creative experiments，并不会自动从模型里弹出一张现成目录。研究者会先构造带强假设的 probe，例如 circular probe 去寻找周期结构；也会对 representation 做 PCA，把高维变

化投影到少数主轴上；还会通过平均或对齐，尽量把具体语义内容洗掉，再看剩下的低维形状是否稳定。这类方法像把一团毛线先按颜色、方向和张力分层，研究者看到的“线圈”并非原始模型直接贴好的标签，而是实验设计和统计投影共同显影出来的结构。

第一类是 successor heads，即后继项注意力头。它们服务于自然顺序：星期、月份、序数、字母、数字等。与归纳头从上下文里复制后继 token 不同，successor head 更像从参数记忆中取出“下一个”。例如模型看到 Monday，可能激活 Tuesday 方向；看到 January，可能激活 February 方向。这里的共性落在“有序集合上的后继运算”，具体词表只是承载这种运算的材料。

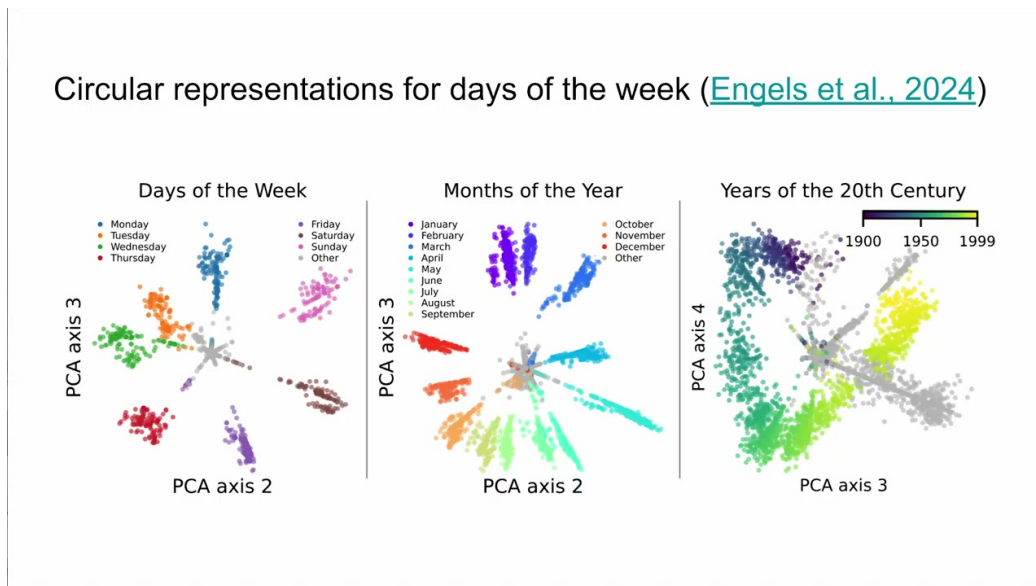


图 10: 周期性概念在表示空间中的环状结构：一周七天、一年十二个月与二十世纪年份都在低维投影中形成可重复的相位模式，说明某些计算基元可以把离散序列压缩成几何结构。¹⁰

第二类是位置与列表检索。讲者提到扩展的位置表示：模型内部可能形成相对位置表征，供 previous-token head 等组件使用，这种表征与硬编码的位置嵌入或 RoPE 分属不同对象。随后他把这条线索连接到 line-breaking 相关论文，字幕中没有足够细节支撑论文细节，只能保留为“近期工作发现了相似位置概念”的线索。

第三类是低维几何结构。抑制头研究里，讲者说他们对头的权重做奇异值分解，发现抑制信号高度集中在奇异值谱的少数方向上。SVD 的标准形式是：

$$W = U\Sigma V^{\top}$$

其中各符号的含义是：

- W ：被分析的权重矩阵，例如某个注意力头的输出相关矩阵。
- U, V ：两组奇异向量，给出输入侧和输出侧的主要方向。
- Σ ：奇异值对角矩阵，表示每个方向携带信号的强弱。
- $V^{\top}$ ： V 的转置。

在讲者描述中，抑制头输出的关键信号近似是一维的：沿残差流中的某个方向写入一个标量强度，再传给后面的 mover head。多个抑制头共同形成一个低秩子空间；他们还把列表项目检索画成

¹⁰ 视频时间区间：00:48:39–00:49:55。

三维几何图景，尽管实际子空间可能更高维。列表中间项容易混在一起，这为“lost in the middle”一类错误提供了机制解释：模型在小子空间中导航不够精确，导致中间项目的可分性下降。

SVD 与子空间在本段的作用

SVD 在本段的作用，是把“很多 head 写入很多向量”压缩成少数方向。若抑制信号主要集中在一个或几个奇异向量上，研究者就能把复杂电路改写成更可检查的子空间问题。

第四类是周期与旋转结构。字幕中提到 arithmetic helix、星期和月份的 circular representations，以及听众对周期系统的补充：如果星期构成周期为 7 的系统，Monday 接在 Sunday 后面，与其他相邻关系一样自然，那么二维正余弦投影会形成环。可用最简单的编码表达：

$$\phi(k) = \left(\cos \frac{2\pi k}{T}, \sin \frac{2\pi k}{T} \right)$$

其中各符号的含义是：

- k ：序列中的第 k 个位置，例如一周中的某一天。
- T ：周期长度，例如星期系统中 $T = 7$ 。
- $\phi(k)$ ：把离散周期项映射到二维圆环上的表示。

这个公式是解释性重构，用来说明为什么周期概念自然落到圆上；讲者没有在字幕中完整推导它。它与 successor head 形成互补：successor head 关心“下一个”，循环表征关心“过了末尾以后回到开头”。

第五类是 binding vectors。讲者把它和抑制头、序列位置选择联系起来：模型可能给序列中的 token 打上某个空间方向标签，后续组件再寻找带有三角形标签、方形标签或其他标签的项目。这是一种“绑定”机制，类似给仓库货架贴颜色胶带：后续检索不必记住完整货物内容，只要寻找对应胶带即可。

最后是 fine-tuning 与 cross-coding。讲者多次强调相关证据有限，但倾向于认为微调常常强化已有电路，并不必然从零发明全新电路。他提到 SAE 特征在 base model 与 post-trained model 之间可能转移良好，也提到可能是 Anthropic cross-coding / cross-layer transcoder 相关工作，用于比较 base 与 fine-tuned 模型。更一般地说，neurons、SAE features、cross-layer transcoder features 可以被看作比完整电路更细的分析层级：神经元给出单个坐标或局部方向，SAE feature 试图把叠加表示拆成更稀疏的语义/功能特征，cross-layer transcoder feature 则试图追踪跨层的功能变换。字幕中这些论文名和作者名有明显 ASR 破损，因此本文只保留研究线索，不把它写成已核验引用。

关于 fine-tuning 的证据边界

本段只能支持温和说法：若干工作暗示微调可能更多是在既有结构上调权重、增强或压低某些路径。它不能支持“微调绝不产生新机制”这种绝对命题，也不能支持“RL 只是在表面包装模型”这种未经限定的结论。

5.6 本章小结

本章把长问答压缩成五条限制。

第一，人工干预证明了因果链，却没有直接给出可部署编辑。研究者在内部电路上做了小而精确的修复，核心证据是下游电路随之恢复；这种证据很有价值，但它依赖研究者已经知道要修哪里。

第二，行为正确和机制正确需要分开。模型可以通过列表选择、位置抑制和候选搬运完成任务，这种实现未必符合人类对语义解析的期待。机制可解释性的锋利之处，正是把这种差异暴露出来。

第三，规模化仍是开放问题。大模型可能有更多冗余、更分布式的机制和更抽象的特征；小模型电路提供了研究入口，却不能自动代表更大模型的内部结构。

第四，机制洞见可以反哺训练数据设计，但路径是诊断性的：先定位表示性弱点，再构造 targeted examples 去施压。它也保留了理解模型本身的科学价值。

第五，更多 motif 线索显示模型内部存在可复用结构的强证据：后继项头、扩展位置表示、SVD 子空间、周期/旋转结构、binding vectors，以及微调前后可能持续存在的特征。这些线索共同支持计算基元这一研究方向，同时也提醒读者：当前证据仍受任务分布、模型规模、人工实验设计和字幕/论文核验质量限制。

本章的压缩结论

机制迁移的正确读法是“找到可复用结构的证据”，不能升级成“获得通用语义算法的证明”。真正难的问题在后面：如何把人工反事实干预变成可泛化发现，如何把小模型电路升级为大模型里的子空间或特征族，如何区分行为上有效的捷径和语义上稳健的机制。

6 从电路到权重空间：共享结构与专属结构

6.1 为什么只做电路分析还不够

前面几章把 transformer 写成了信息流图：节点是注意力头、MLP、残差流位置等组件，边是信息如何从一个组件传到另一个组件。这个视角适合解释 IOI 电路和彩色物体任务中的上下文检索，也适合做路径 patching、人工干预和因果中介分析。讲者在本段转换话题时给出一个很直接的判断：定义结构本身很难。也就是说，找到一条可读的电路，只解决了“这次前向传播里哪些组件在起作用”的问题，还没有解决“参数本身怎样把大量任务的共用能力和少数样本的专属记忆混在一起”的问题。

权重空间提供了另一个坐标系。电路分析沿着激活路径追问“信息从哪里流到哪里”，权重空间分析沿着参数方向追问“如果把某些权重方向拿掉、增强或投影出去，哪些行为会变化”。这就像观察城市交通：电路分析看某一天车流经过哪些路口，权重空间分析看道路网络本身哪些路段承担全城通勤，哪些路段只服务一小片社区。前者更贴近一次计算过程，后者更贴近模型长期存储下来的结构。

本章转向

本章的核心转向是：把“可复用机制”从激活路径上的电路，推广到权重空间中的方向。共享结构指很多数据、任务或行为共同依赖的参数方向；专属结构指少数样本、少数任务或具体记忆更依赖的参数方向。讲者明确把它写成连续谱，而非硬边界。

这里要格外小心一个概念边界。权重空间中的“方向”并不等于某个单独权重元素，也不等于某个自然语言标签。对于有 n 个参数的模型，权重空间可以想成 $\mathbb{R}^n$ ，每个点是一整套参数，每个方向是一种同时改变许多参数的方式。一个方向可能跨层、跨矩阵、跨多个注意力头或 MLP；这也是讲者提醒的难点：真实结构可能分布在层、组件和序列位置之间，attention head 只是某些实验里刚好好用的分析层级。

¹¹ 视频时间：00:58:10–00:58:55。

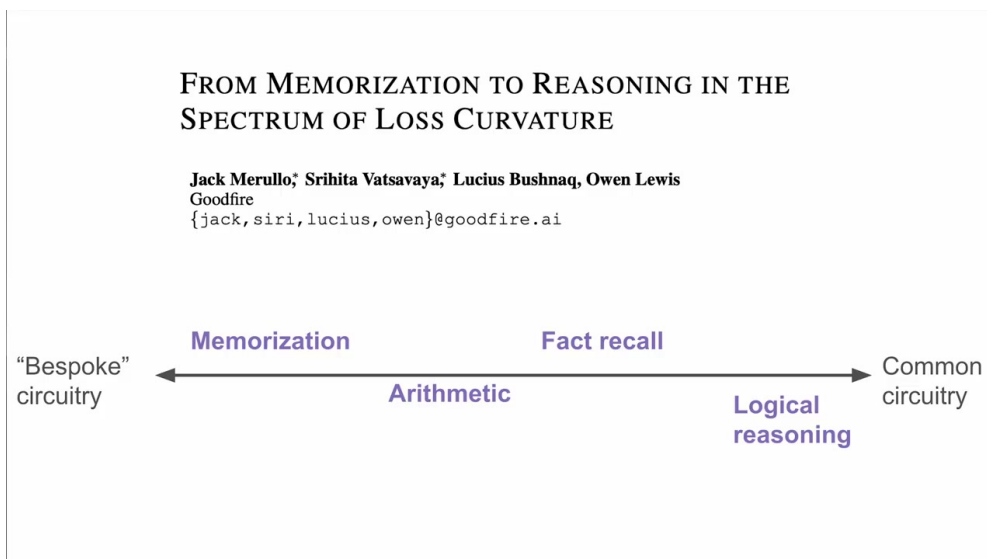


图 11: 损失曲率谱系: 左侧是更专属的记忆化电路, 右侧是更共享的逻辑推理电路, 事实回忆与算术位于中间区域。¹¹

6.2 共享电路到专属记忆的谱系

讲者把后续 Goodfire 工作描述为“直接分解权重”, 目标是区分共享结构和专属结构。本文按术语表固定写作: 共享结构和专属结构; 英文里有时会写成 shared structure 与 unique / bespoke structure。共享结构更像很多任务都会借用的道路, 例如逻辑推理、常见上下文检索、归纳头这类反复出现的计算基元; 专属结构更像只服务少量具体点的窄路, 例如某段训练文本的逐字记忆、某些算术事实、某些具体样本触发的权重方向。

这条谱系可以写成三个层次:

1. 行为层: 逻辑推理、事实回忆、算术、逐字记忆等 benchmark 响应不同。
2. 机制层: 某些行为更依赖共同电路, 某些行为更依赖少数局部方向。
3. 参数层: 这些差异表现为权重空间中方向的重要性、脆弱性和可编辑性不同。

讲者给出的直觉是: 如果移除那些“不常被多数数据使用”的方向, 逻辑推理类任务受到的影响较小, 逐字记忆和部分算术更容易受损。这个说法需要按实验语境理解。它不意味着推理完全没有参数记忆, 也不意味着记忆化只是噪声。更准确的表述是: 不同任务在共享结构与专属结构之间的依赖比例不同, 编辑权重方向后可以观察到一个从稳健到脆弱的行为谱。

谱系边界

不要把“共享结构”和“专属结构”理解成道德化分类。共享不自动代表更高级, 专属也不自动代表无用。对模型来说, 某个罕见事实、训练样本或特殊格式也可能需要专属结构来保存; 问题在于研究者能否识别这种依赖, 并判断它是期望能力、隐私风险、过拟合痕迹, 还是可被选择性编辑的目标。

讲者还提到记忆化的定义可以有严格和宽松版本。严格版本接近贪婪解码时逐字生成一段较长的训练后缀; 宽松版本则允许用编辑距离衡量生成内容与训练片段的接近程度。这个区分很重要, 因

为“记忆化”不应被压成只有 0 和 1 的标签。它更像光谱仪读数：完全复现训练文本是强信号，近似复现是弱一些但仍值得警惕的信号。

6.3 Fisher / Hessian 近似的直觉

先给直觉。损失曲率衡量的是：沿着某个权重方向轻轻推一下，模型损失会上升得多快。如果某个方向像全城主干道，许多样本都会经过它，那么扰动它会让许多样本的 loss 都有变化，平均起来曲率较高；如果某个方向只服务一个记忆化样本，那么它可能只在该样本附近很尖锐，对大多数数据却很平坦。讲者在白板上用“flat basin”和“sharp”来讲这个图像：一个方向对多数数据平坦，对少数记忆点尖锐；另一个方向对许多数据都有可见影响。

用二阶近似写出来，设模型参数为 θ ，在当前参数 θ_0 附近沿方向 d 走一小步 ϵd ，损失可以局部近似为：

$$L(\theta_0 + \epsilon d) \approx L(\theta_0) + \epsilon \nabla_{\theta} L(\theta_0)^{\top} d + \frac{1}{2} \epsilon^2 d^{\top} H d.$$

符号说明：

- $L(\theta)$ ：模型损失，表示参数为 θ 时在某个数据分布或样本集合上的 loss。
- θ_0 ：当前模型参数，也就是做局部分析的起点。
- d ：权重方向，表示参数空间里准备扰动、编辑或投影的方向。
- ϵ ：很小的标量步长，控制沿 d 移动的幅度。
- $\nabla_{\theta} L(\theta_0)$ ：loss 对参数的一阶梯度，描述当前位置最直接的上升方向。
- $H = \nabla_{\theta}^2 L(\theta_0)$ ：Hessian 矩阵，即 loss 对参数的二阶导数矩阵，描述局部损失曲率。
- $d^{\top} H d$ ：方向 d 上的二阶曲率。值越大，沿这个方向移动越容易让 loss 快速变大。

如果模型已经在某个局部区域训练得较稳定，一阶项在平均意义上往往不再是最主要的分析对象，二阶项 $d^{\top} H d$ 就成了判断方向重要性的核心量。Hessian 矩阵很大，直接计算通常不可行，所以讲者转向 Fisher 信息矩阵。Fisher 信息矩阵可以看成用梯度统计构造出的曲率近似；在交叉熵训练的模型里，它经常被当作损失地形曲率的可计算替身。字幕在这一段有明显 ASR 失真，因此这里按讲者白板上的可见数学直觉和标准近似关系来解释，而不声称复现完整论文推导。

一个常见写法是：

$$F = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla_{\theta} \ell(x, y; \theta) \nabla_{\theta} \ell(x, y; \theta)^{\top}], \quad d^{\top} F d \approx d^{\top} H d.$$

符号说明：

- F ：Fisher 信息矩阵，用梯度外积的期望来近似参数方向的重要性或曲率。
- (x, y) ：数据样本与目标标签；在语言模型里， x 可理解为上下文， y 可理解为目标 token 或目标序列。
- $\mathcal{D}$ ：取期望的数据分布或评估样本集合。
- $\ell(x, y; \theta)$ ：单个样本上的损失。

- $\nabla_{\theta}\ell(x, y; \theta)$: 单样本 loss 对参数的梯度。
- $\nabla_{\theta}\ell(x, y; \theta)^{\top}$: 该梯度的转置。
- $\mathbb{E}_{(x, y) \sim \mathcal{D}}[\cdot]$: 对数据分布或样本集合求平均。
- $d^{\top} F d$: Fisher 信息矩阵给出的方向 d 上的近似曲率。
- H : Hessian 矩阵, 完整二阶曲率对象; 这里用 F 近似它的方向性作用。

Hessian 与 Fisher

Hessian 矩阵像真实地形测量: 它直接描述 loss 山谷在每个方向上的弯曲程度。Fisher 信息矩阵像用脚印密度估计地形: 它统计梯度如何在数据上变化, 很多时候能给出可用的曲率近似。二者关系紧密, 但不能随意互换; 在模型、损失、采样分布和局部最优假设不同的时候, 近似质量会变化。

这个数学工具与共享结构、专属结构的关系在于“平均”。如果一个方向被很多数据共同使用, 它会在许多样本的梯度统计中反复出现, 平均后仍然显眼; 如果一个方向只对少数记忆样本尖锐, 它在全数据平均中可能落到较低曲率或更局部的成分里。讲者承认这里有多步连接: 从 loss 曲率到记忆化, 再到共享/专属结构, 中间依赖前人关于曲率与记忆的工作, 也依赖具体实验设计。

6.4 K-FAC 风格的矩阵分解

接着看讲者白板上的线性层直觉。设 transformer 中某个线性层接收激活向量 a , 用权重矩阵 W 产生输出。反向传播时, 输出侧会有反向梯度向量 g 。对一个线性层来说, 单样本权重梯度通常具有外积结构: 输入侧给出“读入了什么”, 输出侧梯度给出“该输出方向对 loss 有多敏感”。这正是 K-FAC 能简化 Fisher 信息矩阵的入口。

最基本的线性映射是:

$$y = Wa.$$

符号说明:

- a : 激活向量, 线性层输入侧的前向传播值。
- W : 权重矩阵, 线性层的参数。
- y : 线性层输出向量。

若输出侧反向梯度为 $g = \nabla_y \ell$, 则权重矩阵的单样本梯度可写成:

$$\nabla_W \ell = g a^{\top}.$$

符号说明:

- $\nabla_W \ell$: 单样本损失对权重矩阵 W 的梯度。
- g : 反向梯度向量, 表示 loss 对线性层输出 y 的敏感性。
- $a^{\top}$: 激活向量 a 的转置。

- $ga^\top$: 外积, 形状与 W 相同, 表示每个输出梯度方向和每个输入激活方向的组合。

K-FAC 的关键近似是: 避开直接构造巨大 Fisher 信息矩阵的昂贵路线, 分别统计激活和梯度, 再用 Kronecker 积组合它们。讲者在白板上说到对 a 做协方差统计, 对 g 做同样统计, 然后利用 K-FAC 论文中的事实, 把权重矩阵的 Fisher 结构近似为这两个统计矩阵的 Kronecker 积。

下面的 F_W 指权重矩阵 W 对应的 Fisher block, 而非整模型的完整 F 。可见白板数学直觉可以写成:

$$A = \mathbb{E}[aa^\top], \quad G = \mathbb{E}[gg^\top], \quad F_W \approx A \otimes G.$$

符号说明:

- A : 激活协方差 / 未中心化二阶矩, 统计多个样本中输入激活 a 的主要变化方向。
- G : 梯度协方差 / 梯度二阶矩, 统计多个样本中输出侧反向梯度 g 的主要变化方向。
- a : 线性层输入侧激活向量。
- g : 线性层输出侧反向梯度向量。
- $aa^\top$: 激活向量与自身的外积。
- $gg^\top$: 梯度向量与自身的外积。
- $\mathbb{E}[\cdot]$: 在样本集合上求平均。
- F_W : 针对权重矩阵 W 的 Fisher 信息矩阵近似。
- $\otimes$: Kronecker 积, 把两个矩阵按块状方式组合成更大的矩阵。
- $F_W \approx A \otimes G$: K-FAC 近似, 含义是用两个较小统计矩阵的 Kronecker 积表示巨大曲率矩阵的主要结构。

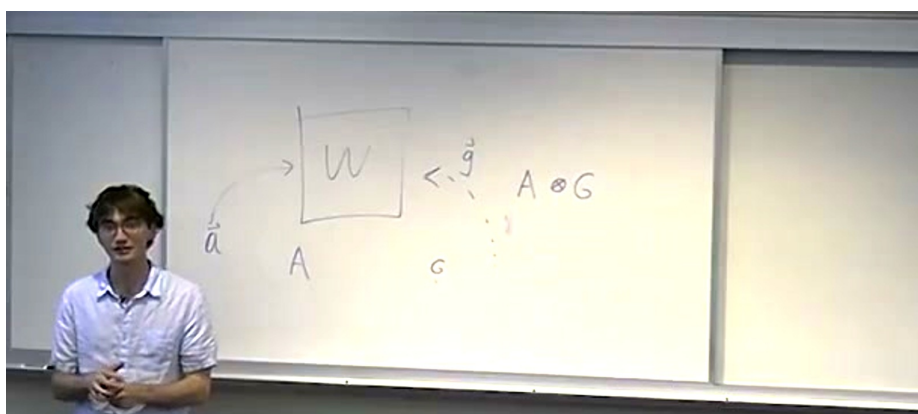


图 12: K-FAC 直觉: 权重矩阵 W 接收前向激活 a , 反向传播给出输出侧梯度 g ; 激活协方差 A 与梯度协方差 G 的 Kronecker 近似用于描述 Fisher 曲率。¹²

¹²视频时间: 01:07:40–01:11:34。

Kronecker 积直觉

Kronecker 积可以类比成“矩阵积木展开”：若 A 记录输入侧哪些激活方向常见， G 记录输出侧哪些梯度方向重要，那么 $A \otimes G$ 就把“输入方向”和“输出方向”的组合铺成权重空间中的方向。它和普通矩阵乘法 AG 不同；普通乘法把维度压合，Kronecker 积把两个结构展开成更大的块结构。

为什么这有助于找权重方向？因为对 A 和 G 做特征分解时，可以得到输入激活的主方向和输出梯度的主方向。讲者在白板上写到，若从 A 中取一个特征向量 u ，从 G 中取一个特征向量 v ，它们的外积可以看成与 W 同形状的一片权重矩阵方向：

$$\Delta W_{u,v} = vu^\top.$$

符号说明：

- $\Delta W_{u,v}$ ：由输入侧方向 u 和输出侧方向 v 组合出的权重编辑量或权重方向。
- u ：来自 A 的特征向量，表示激活空间中的一个主要方向。
- v ：来自 G 的特征向量，表示梯度空间中的一个主要方向。
- $u^\top$ ：方向向量 u 的转置。
- $vu^\top$ ：外积，形成与 W 形状匹配的矩阵方向。

这一步的教育意义很强：K-FAC 把“巨大参数空间里哪里弯曲”这个问题，拆成“输入激活有哪些主要方向”和“输出梯度有哪些主要方向”两个较小问题。它的代价是近似假设。它不会自动告诉研究者某个方向的语义名称，也不会保证每个主方向都可解释；它只提供一个更可计算的坐标系，让选择性编辑和行为测量有了落点。

6.5 投影、编辑与行为测量

得到权重方向后，讲者关心的下一步是编辑。所谓选择性编辑，就是沿某些方向改变权重，或把某些方向从权重中投影出去，然后观察 benchmark 响应。若移除低曲率、较专属的方向后，某些记忆化任务明显变差，逻辑推理受影响较小，就说明这些行为对权重结构的依赖不同。若移除高曲率、共享结构方向后，许多任务同时下降，那就提示这些方向更像共用底座。

投影的直觉是把一个向量拆成“落在目标子空间里的部分”和“剩下的部分”。若 S 是由若干权重方向张成的子空间， $P_S(\Delta W)$ 表示把权重编辑量投影到 S 上，那么移除该子空间分量可写成：

$$\Delta W_{\text{removed}} = \Delta W - P_S(\Delta W).$$

符号说明：

- ΔW ：原始权重编辑量，或准备分析的权重变化。
- S ：目标子空间，由一个或多个权重方向张成。
- $P_S(\Delta W)$ ： ΔW 在子空间 S 上的投影分量。
- $\Delta W_{\text{removed}}$ ：移除子空间 S 分量后的剩余编辑量。

若目标是直接编辑当前权重矩阵，也可以把它写成：

$$W' = W - \alpha P_S(W).$$

符号说明：

- W' ：编辑后的权重矩阵。
- W ：编辑前的权重矩阵。
- α ：编辑强度； $\alpha = 1$ 表示按当前投影定义完整移除该分量，较小值表示温和编辑。
- $P_S(W)$ ：权重矩阵 W 在目标子空间 S 上的投影。
- S ：由待移除或待削弱的权重方向构成的子空间。

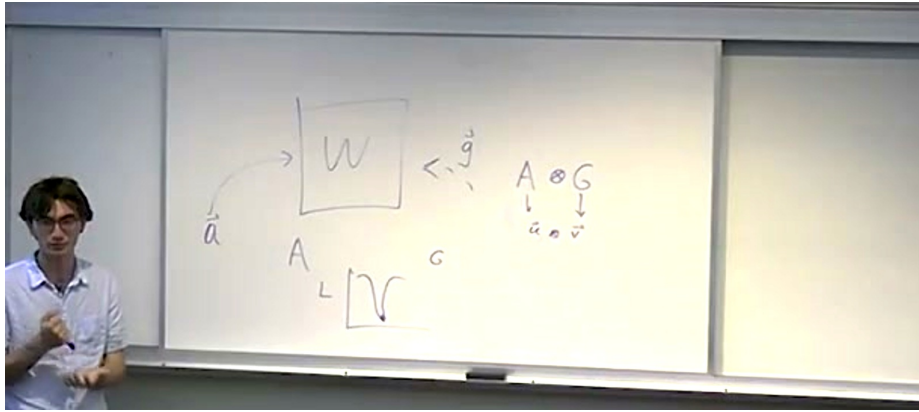


图 13: 从曲率到编辑方向：损失曲率刻画权重方向对数据平均行为的影响，激活主方向 u 与梯度主方向 v 的外积形成可投影、可移除的权重切片。¹³

投影的对象

投影并不等同于“删除某个神经元”。它通常作用于高维方向或子空间，可以跨越许多参数。把投影视为删除一个可命名的部件，会误导读者低估分布式表示的复杂性。更准确的类比是调音台：研究者降低某个频段的能量，再听整首曲子哪些声部发生变化。

行为测量是这套方法的闭环。选择性编辑本身只是一种干预；它是否揭示机制，要看编辑后的 benchmark 响应。讲者的结果被概括成 memorization 与 reasoning 之间的谱：某些编辑让逐字记忆、算术或较脆弱的行为更容易下降，逻辑推理类行为相对稳健。这里的“相对”很关键。它指实验中的比较关系，不应扩写成所有模型、所有推理任务、所有记忆任务的普遍定律。

6.6 本章小结

本章把机制可解释性的坐标系从电路扩展到权重空间。电路分析问：一次前向计算里，哪些组件和路径承担了因果贡献？权重分解问：参数空间里，哪些方向被很多任务共同依赖，哪些方向更服务少数样本或具体记忆？Fisher 信息矩阵和 Hessian 矩阵提供损失曲率视角，K-FAC 用激活协方

¹³ 视频时间：01:12:03–01:16:44。

¹⁴ 视频时间：01:16:51–01:17:14。

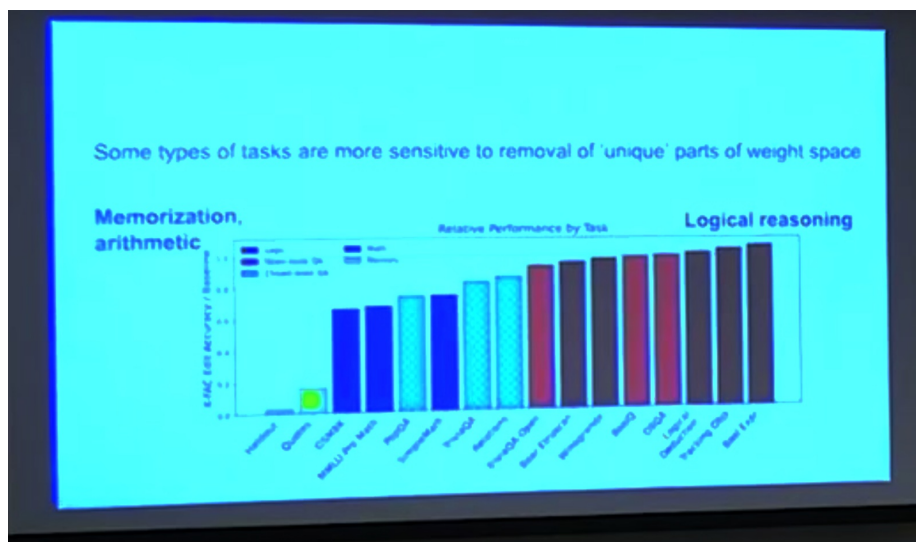


图 14: 选择性编辑后的行为谱：移除更“专属”的权重部分后，记忆化和算术任务下降更明显，逻辑推理任务更稳定。¹⁴

差和梯度协方差的 Kronecker 积近似巨大曲率对象。投影和选择性编辑则把这些方向变成可测试的干预：改掉某些方向，再看 benchmark 响应如何移动。

更压缩地说：电路分析在激活路径上找机制，权重分解在权重空间里找共享结构与专属结构。两者互补。前者更像解释一条计算链路，后者更像测量参数地形中哪些山脊支撑了哪些行为。

7 应用与开放问题：从选择性编辑到科学发现

7.1 选择性编辑与脆弱性谱

选择性编辑最直接的用途是诊断脆弱性。若某类行为在移除低曲率或专属方向后迅速下降，它更可能依赖少数局部结构；若某类行为在同样编辑下保持稳定，它可能有更多共享结构、冗余路径或更分散的实现。这个观察为模型安全和模型治理提供了一种比“整体准确率”更细的测量方式：研究者可以问一个行为依赖多少共享底座，多少专属记忆，多少偶然的脆弱方向。

这也解释了为什么本讲从 colored objects 的人工干预走到权重空间。colored objects 里，研究者修复 inhibition heads 的注意力模式，观察 mover head attention 和错误答案概率如何变化；权重空间里，研究者移除或削弱某些方向，观察 benchmark 响应如何变化。两者都属于反事实干预，只是干预位置不同：一个在激活路径，一个在参数方向。

选择性编辑的定位

选择性编辑的核心价值是把行为依赖拆开：哪些能力靠共享结构支撑，哪些输出靠专属结构保存，哪些任务对某类权重方向特别脆弱。这个拆分本身就是解释性结果；“把模型变好”只是可能出现的工程目标之一。

7.2 长上下文和规模化 circuit tracing

讲者在问答中承认，传统电路分析面对规模化时会遇到两类压力。第一类是计算压力。路径 patching 的朴素版本需要大量 ablation、反事实替换和前向传播；梯度归因可以加速一部分搜索，

但并不自动解决所有解释问题。第二类是理解压力。即使在 70B 级模型或更大模型上能得到一堆归因结果，研究者仍然要解释这些结果代表什么。

更大的挑战来自长上下文。短序列上的电路分析已经有不少工具，但长链条推理、数学竞赛、代码任务和长上下文问答可能涉及成千上万个 token，甚至多条隐含推理链。讲者的担忧很朴素：当前工具可以分析很大的模型在短时间窗口里的行为，却很难消化一个长上下文 roll-out 中连续发生的机制变化。序列维度一旦拉长，搜索空间会膨胀，结果也更难压缩成人能读懂的结构。

规模化的真正压力

规模化并不只是“跑得更快”。真正的问题还有表征层级：attention head 有时是好抽象，有时结构分布在层、token 位置、特征方向和权重子空间之间。若分析层级选错，再多 forward pass 也只能产生更厚的日志。

这给自动化留下空间。讲者提到语言模型本身越来越能辅助科学工作，可以帮助扩展搜索空间、提出候选机制、整理结果。但自动化在这里更像显微镜自动对焦：它能帮助找到可疑区域，不能替研究者完成因果验证和概念命名。机制可解释性的瓶颈，一半在算力与搜索，一半在把结构压缩成可靠概念。

7.3 科学模型中的发现工具

讲者最后把解释性方法推向以 DNA / nucleotide-level model 为主的科学模型例子。核心想法是：模型吞下的数据规模远超单个研究者能手工阅读的范围，它可能学到人类尚未系统整理的信号。解释性工具如果能定位这些内部信号，就可能帮助科学发现。例如在 DNA 模型中，某些激活空间方向或 SAE 特征可能对应疾病相关 genetic marker、基因组注释线索，或某种尚未被充分命名的序列模式。

字幕中这一段有多处 ASR 不确定：机构名大概率指 Arc Institute，具体 marker 名称也没有可靠转写。因此本文保留谨慎措辞：讲者提到 Goodfire 与相关科学机构合作，在 nucleotide-level model 上训练 SAE，并用激活空间方向帮助注释基因组序列；具体论文题名、机构拼写和 marker 名称仍需独立资料核验。

科学发现的证据门槛

科学发现里的“特征”要经过外部验证。SAE 特征或激活空间方向最多提示“模型内部有一个可复现信号”，它还没有达到生物学结论的证据标准。可靠路线通常是：发现方向，命名假设，检查对应序列或样本，做统计验证，再交给领域实验或文献证据校准。

这个应用也反过来解释本讲题目“计算基元”的野心。若模型内部的计算基元只是解释 toy task，它的价值有限；若同类方法能帮助研究者从海量科学数据中发现规律，它就成为一种新的实验仪器。仪器的读数需要校准，读数本身也可能有噪声，但它扩大了人能观察到的结构范围。

7.4 本章小结

应用价值来自两个方向。第一，诊断模型依赖了什么：通过选择性编辑和 benchmark 响应，判断行为对共享结构、专属结构、低曲率方向或高曲率方向的依赖。第二，发现训练数据中人类难以手工穷尽的结构：通过 SAE 特征、激活空间方向和权重空间方向，把模型学到的潜在规律转化为可检查的科学假设。

开放问题同样清楚：长上下文如何分析，规模化 circuit tracing 如何解释，权重方向如何命名，选择性编辑如何避免副作用，科学模型中的内部方向如何通过外部证据验证。讲者给出的答案还不完整；这恰好说明机制可解释性仍处在工具快速成形、概念尚未完全稳定的阶段。

8 总结与延伸

8.1 讲者的收束

讲者的实质性收束有三层。第一，前面那些电路例子显示可复用机制的强线索：归纳头、IOI 电路、colored objects 的上下文检索，都指向跨任务共享组件。第二，权重分解提供了另一种路线：通过损失曲率、Fisher 信息矩阵、K-FAC 和投影，把共享结构与专属结构放进同一个权重空间谱系里测量。第三，解释性工具的最终价值可能超出语言模型本身，进入 DNA / nucleotide-level 科学模型，用内部方向帮助研究者发现人类尚未显式整理的信号。

例行感谢和超时收尾不属于教学内容，因此这里不展开。保留下来的重点是“tools for processing data with these models and novel discovery”：机制可解释性既是解释模型的工具，也是处理大规模数据、提出新科学假设的工具。

8.2 全讲核心链条

全讲可以压缩成一条链：

motif $\rightarrow$ circuit $\rightarrow$ intervention $\rightarrow$ transfer $\rightarrow$ weight direction $\rightarrow$ selective edit $\rightarrow$ application.

符号说明：

- motif：计算 motif / 计算母题，指跨任务反复出现的机制模式。
- circuit：电路，指承载某个预测或中间计算的组件和路径集合。
- intervention：干预，指路径 patching、注意力修复、投影或权重编辑等反事实操作。
- transfer：迁移，指机制从 IOI 到 colored objects 等不同任务中的复用。
- weight direction：权重方向，指权重空间中可被测量、投影或编辑的参数方向。
- selective edit：选择性编辑，指沿特定方向修改模型并观察行为响应。
- application：应用，指模型诊断、记忆化分析、规模化 circuit tracing 和科学发现。

这条链的每一步都在回答同一个问题：模型内部有没有比表面任务更稳定的结构？归纳头给出第一个样板，IOI 电路给出可检验的信息流，colored objects 让跨任务复用从猜想变成可检验假说，权重空间分解把“共享”与“专属”推进到参数层，选择性编辑把结构假设变成行为测试，科学模型应用把解释性推向发现工具。

8.3 概念压缩

第一组对偶是“行为层面的任务”与“机制层面的电路”。任务是外部题目，例如识别间接受词或回答物体颜色；电路是内部实现，例如重复 token 头、抑制头、搬运头和残差流路径。两个任务表面不同，可能共享一部分电路；一个任务表面简单，也可能由多个机制拼接完成。

第二组对偶是“激活路径上的因果贡献”与“权重空间中的共享方向”。激活路径描述一次运行中信息如何流动，权重方向描述参数地形里哪些变化会影响哪些行为。前者像拍摄交通流的短片，后者像测绘道路和地形。两种视角配合使用，才能从“这次模型怎样算”走向“模型把这种算法存在哪里”。

第三组对偶是“人工反事实干预”与“可泛化的结构发现”。人工干预有时显得合成，比如直接修复 inhibition head 的注意力模式；但它的作用是建立因果证据。可泛化结构发现要求更高：研究者希望方法能在新任务、新模型、新科学数据中自动或半自动找到可解释方向。当前领域已经有强信号，也有明显缺口。

本讲压缩结论

本讲最值得带走的一句话是：机制可解释性想交付的成果，不能停在单个案例的漂亮故事上；更高标准是可复用、可干预、可迁移、可测量的计算基元。电路、Fisher 信息矩阵、K-FAC、Kronecker 积、投影和选择性编辑，都是为了把这些基元从直觉变成可操作对象。

8.4 后续阅读和检查清单

后续阅读应优先回查五类材料。第一，归纳头论文，用来理解上下文复制和 in-context learning 的最小机制。第二，IOI paper，用来核对 mover head、inhibition head、duplicate token head 等组件名称和因果路径。第三，K-FAC 论文，用来校准 Fisher 信息矩阵、Kronecker 积和曲率近似的数学细节。第四，Goodfire / Arc Institute 的 DNA 或 nucleotide-level model 相关工作，用来确认讲者最后提到的机构、模型、SAE 特征和 marker 名称。第五，近期 circuit tracing 和规模化归因工作，用来理解讲者所说的长上下文、自动化和结果解释难题。

阅读本讲时，还应带着四个检查问题：

1. 图 F13-F16 是否真实对应本章文字，尤其是白板公式和 benchmark 谱系是否清晰可读；F17 已因视觉复核未采用。
2. Fisher/Hessian/K-FAC 公式是否与可见白板和可采用的论文背景一致，避免把标准教材推导误写成讲者逐字内容。
3. “共享结构 / 专属结构”是否始终写成谱系，避免硬二分。
4. DNA / nucleotide-level 应用中的机构名、paper、marker 名称是否已复核；若无法复核，应保留 ASR 不确定提示。